Data Pipelines Introduction

Data Pipelines Introduction

- **Automation:** Pipelines are typically automated, ensuring data is processed without manual intervention.
- **Scalability:** They are designed to handle large volumes of data as businesses grow.
- **Real-time Processing:** Many pipelines support real-time data flows, enabling instant insights.
- **Flexibility:** Data pipelines can integrate with various data sources and destinations, such as databases, cloud storage, and APIs.

Data Pipelines Introduction

- Why Are Data Pipelines Important?
  - **Data Integration:** They connect disparate data sources into a unified system.
  - **Efficiency:** Automation reduces manual effort and speeds up the process.
  - **Consistency and Reliability:** Well-designed pipelines ensure that data is processed accurately and consistently.
  - **Decision-Making:** By delivering processed data to analytical systems, they enable informed decisions.

Data Pipeline Architecture

Data Pipeline Architecture

Considerations for Data Pipeline Architecture:

- **Modularity:** Breaking down the pipeline into discrete, reusable components.
- **Scalability:** Designing systems that can handle increasing data volumes with ease.
- **Fault Tolerance:** Ensuring the pipeline can recover from failures without data loss.
- **Real-Time and Batch Support:** Balancing the needs for immediate data processing and periodic bulk processing.
- **Data Quality:** Incorporating mechanisms to validate, clean, and standardize data at every stage.

Data Pipeline Architecture

Importance of Data Pipeline Architecture:

- **Efficiency:** A strong architecture reduces processing time and resource consumption.
- **Maintainability:** Modular and well-documented architectures make pipelines easier to update and troubleshoot.
- **Scalability and Flexibility:** A well-designed architecture can grow with your data needs and adapt to new requirements.
- **Cost Optimization:** Effective architectures minimize unnecessary resource usage, reducing operational costs.

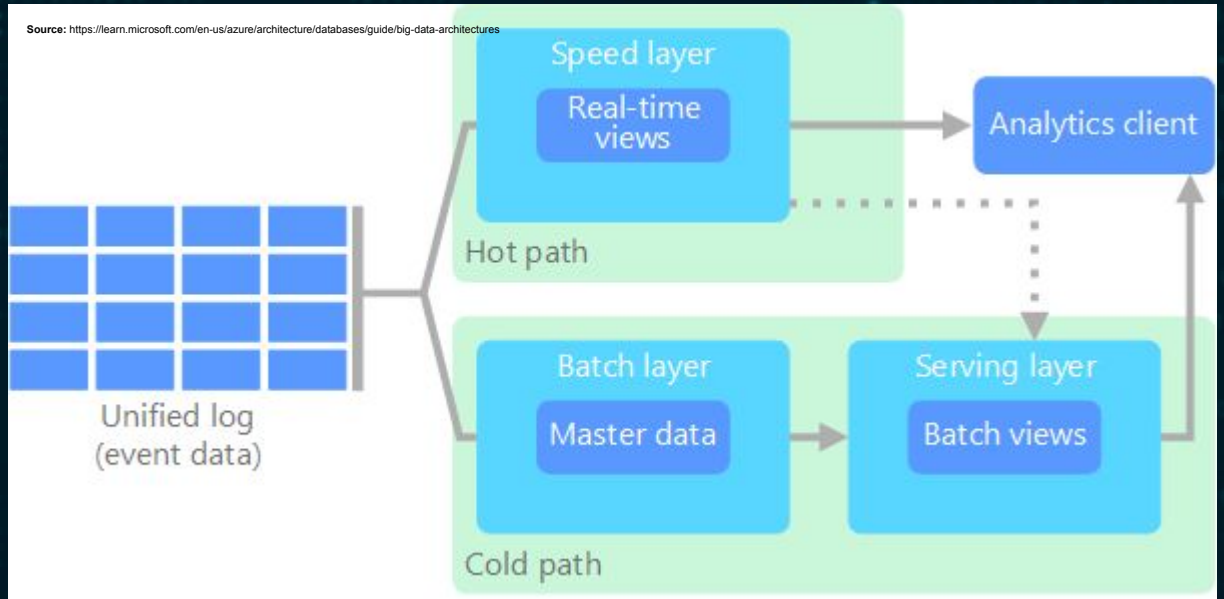
Data Pipeline Architecture

Examples of Data Pipeline Architecture

- **ETL (Extract, Transform, Load):** Data is extracted from sources, transformed into a usable format, and loaded into a destination system. This traditional model is widely used for batch processing.
- **ELT (Extract, Load, Transform):** Data is extracted and loaded into a data warehouse or lake first, then transformed. This approach is popular in cloud-native environments.
- **Lambda Architecture:** Combines real-time and batch processing for low-latency insights and comprehensive historical analysis.
- **Kappa Architecture:** Focuses exclusively on real-time streaming, eliminating the batch layer for simpler use cases.
- **Event-Driven Architectures:** Triggers pipeline processes based on specific events, enabling high responsiveness.

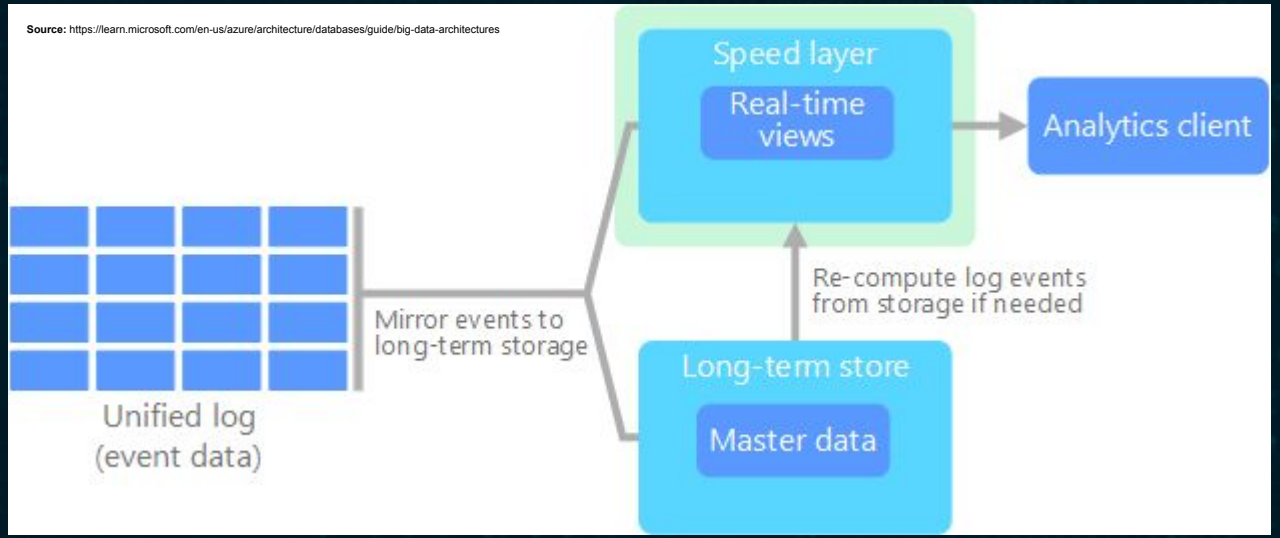
Data Pipeline Architecture

- Lambda Architecture



Data Pipeline Architecture

- Kappa Architecture



ETL vs. ELT

ETL vs ELT

	ETL	ELT
Process Flow	Extract → Transform → Load	Extract → Load → Transform
Transformation Location	Data is transformed in a staging server or ETL tool before loading into the destination.	Data is loaded into the destination first, and transformation occurs within the destination system.
Destination Type	Traditionally used with on-premises databases or data warehouses.	Commonly used with modern cloud-based data warehouses (e.g., Snowflake, BigQuery, Redshift).
Performance	Transformation is limited by the compute resources of the ETL server.	Leverages the scalability and performance of the destination system for transformation.
Data Volume Handling	Better suited for smaller or moderate data volumes due to transformation bottlenecks.	Ideal for large-scale data volumes as transformations use the power of cloud storage and compute.

Designing a Data Pipeline

Designing a Data Pipeline

Scalability and Performance:

- Ensure the pipeline can handle increasing data volumes, velocity, and variety without degradation in performance.
- Design for parallel processing and optimize for low-latency data flow where necessary.

Data Quality and Validation:

- Implement mechanisms for data cleansing, validation, and enrichment at various stages to ensure high-quality, accurate data.
- Include logging and monitoring to detect anomalies or failures.

Resilience and Fault Tolerance:

- Build robust error-handling and retry mechanisms to recover from failures.
- Design for fault isolation and ensure the pipeline can gracefully handle partial failures without affecting downstream processes.

Security and Compliance:

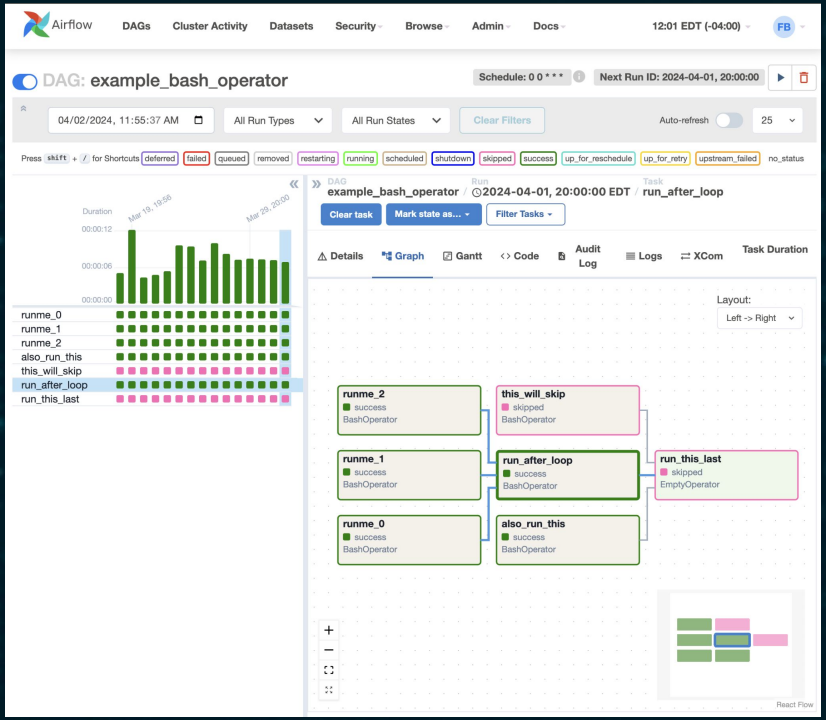
- Protect data at rest and in transit using encryption and access control mechanisms.
- Ensure compliance with regulatory requirements (e.g., GDPR, CCPA) and implement audit trails for transparency.

Overview of Data Pipelines and Use Cases

What Are Data Pipelines?

- Data pipelines are automated workflows that extract, transform, and load (ETL) data from various sources to destinations.
- They ensure efficient, scalable, and reliable data movement and processing.
- Manage dependencies, alert on failures, improve trust in data

Overview of Data Pipelines and Use Cases



Overview of Data Pipelines and Use Cases

- Business Intelligence and Analytics
  - Generate actionable insights from enterprise data.
  - Example: Sales performance dashboards.
- Real-Time Monitoring
  - Monitor systems or business metrics in real-time.
  - Example: Server performance monitoring.
- Data Integration
  - Combine data from multiple sources for a unified view.
  - Example: Customer 360-degree profiles.
- Machine Learning
  - Prepare and serve data for ML model training and predictions.
  - Example: Personalized recommendation systems.

DAGs and Tasks

DAGs and Tasks

DAGs Directed Acyclic Graphs

- Directed - Only move in one direction
- Acyclic - Don’t have cycles (permanently repeating steps)
- Graph - Series of Nodes connected by Edges

DAGs and Tasks

DAG example

1. Wait for CSV to land from external vendor
2. Perform transformations on the data
   a. If Category == “Fruits” SUM
   b. If Category == “Vegetables” AVG
3. Insert both sets of data into a SQL database
4. Perform a Data Quality check

Id	Name	Color	Price_per_kg	Category
1	Apple	Red	3.5	Fruits
2	Banana	Yellow	1.2	Fruits
3	Orange	Orange	2.8	Fruits
4	Carrot	Orange	1.8	Vegetables
5	Broccoli	Green	2.5	Vegetables
6	Potato	Brown	0.9	Vegetables

DAGs and Tasks

Although this is a simple example there are a lot of contingencies that this pipeline takes into account:

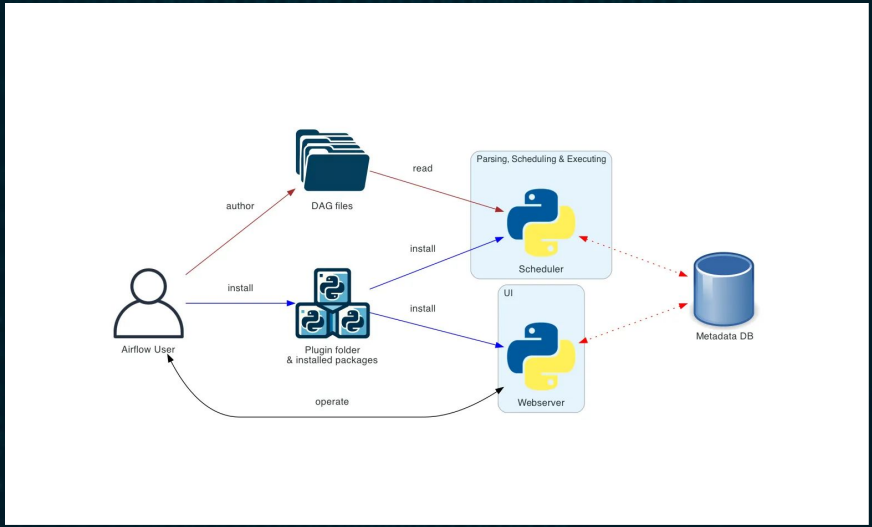
- We only run the pipeline if the data for today is available
- Split up our transformation logic based on what our inputs are
- Only perform the SQL insertion after we have both sets of data ready
  - There can be delays at any point in this pipeline for any reason, instead of erroring out or inserting incomplete data, our pipeline will just hold formation until all prerequisites are fulfilled
- Remove data that doesn't meet a quality threshold, this can be important in helping build trust in your organization's data be catching errors and problems before they are surfaced to end users

Airflow Architecture

Airflow and Architecture

There are five major components to this diagram

- DAG Files
  - Authored by the Airflow user
  - Read by the Scheduler for parsing, scheduling, and execution
  - While this is oftentimes part of the rest of the Airflow application, in many prod environments, the DAG files are stored in a separate system and managed using a version control software like GitHub and a Continuous Integration / Continuous Deployment pipeline
- Scheduler
  - Handles parsing, scheduling, and execution of DAGs
  - Interacts with the Metadata Database to track state and store information
  - This is possibly the most complex part of Airflow as there are several ways you can split this system apart based on your usecases. For the sake of this course we'll stick with this simple architecture
- Webserver
  - Provides the UI for the Airflow system
  - Installed and operated by the Airflow user
- Plugin Folder & Installed Packages
  - Contains plugins and installed packages for additional functionalities
  - Installed by the Airflow user and used by both the Scheduler and Webserver
- Metadata Database
  - Stores the metadata for DAGs, tasks, and other components
  - Interacts with both the Scheduler and Webserver to maintain state and track system operations
  - This is one of the most important parts of Airflow's architecture as it maintains the state between all the components of the system. If you start experiencing a slowing Airflow deployment then it might be time to clean up your metadata database



Airflow Pools and Triggerers

Airflow Pools

- **Purpose:** Manage and limit the number of concurrent tasks across the entire Airflow environment.
- **Definition:** A pool is a resource in Airflow that can be used to control the parallelism of task execution.
- **Usage:** You assign tasks to a pool, and those tasks can only run if there are available slots in the pool.
- **Default Pool:** Airflow comes with a default pool called `default_pool` which has a set number of slots that all tasks use unless explicitly assigned to another pool.
- **Custom Pools:** You can create custom pools to allocate resources to specific types of tasks (e.g., a pool for I/O-bound tasks or a pool for heavy computational tasks).
- **Slot:** A slot represents a single unit of capacity in a pool. A task consumes one slot from the pool while it's running.

Airflow Triggerers

- **Purpose:** Efficiently handle large volumes of external event-based triggers or sensors in an Airflow DAG without blocking worker resources.
- **Definition:** Triggerers are lightweight components that manage and trigger deferred tasks or event-based sensors asynchronously.
- **Usage:** Useful for tasks that wait for an external event (like a file arrival) without consuming worker slots, reducing resource usage.
- **Example:** Triggering a task when a file is uploaded to an S3 bucket without continuously polling and occupying a worker.

Airflow Operators

Airflow Operators

- While a DAG and Task are a bit abstract (how you define a task is entirely up to what you consider a “unit of work”), Operators are much more concrete, they’re what allow us to interact directly with our data. Examples of Operators include:
  - **PythonOperator -** Executes a Python function which could theoretically allow you to do anything
  - **BashOperator -** Executes a bash command or script, can help you directly interact with computers
  - **SimpleHTTPOperator -** Performs an HTTP request, useful for interacting with RESTful APIs
  - **S3KeySensor -** Will activate when a certain file becomes available in AWS S3
- List of all operators: https://www.astronomer.io/docs/learn/what-is-an-operator

```
S3ToRedshiftOperator(
        task_id="load_to_redshift",
        s3_bucket="{{ var.json.aws_configs.s3_bucket }}",
        s3_key="{{ var.json.aws_configs.s3_key_prefix }}" + f"/{CSV_FILE_PATH}",
        schema="PUBLIC",
        table="{{ var.json.aws_configs.redshift_table }}",
        copy_options=["csv"],
    )
```

Airflow Hooks

Airflow Hooks

A Hook in Airflow is an abstraction layer that provides a standardized way to interface with external systems or services. Hooks handle the connection and interaction with these systems, abstracting away the details of authentication, connection parameters, and basic operations. They are usually used within Operators to perform actions like connecting to a database, interacting with cloud services, or sending HTTP requests.

- **PostgresHook:** Manages connections to a PostgreSQL database, allowing you to execute SQL queries.
- **S3Hook:** Interacts with Amazon S3, allowing you to upload, download, or manage files in an S3 bucket.

Operators encapsulate a full unit of work such as moving a file from a filesystem to a database. A hook is one aspect of that work, specifically connecting the the systems in question to perform the data transfer.

Advanced Data Pipeline Concepts

Advanced Data Pipeline Concepts

1. Architecture Design
    a. Batch vs. Streaming: Choose based on data timeliness requirements.
    b. Decoupling: Use message queues (e.g., Kafka, NATS) or cloud-native event systems for loosely coupled components.
    c. Modularity: Design reusable, modular components for flexibility.
2. Tool Selection
    a. Orchestrators: Tools like Apache Airflow, Dagster, Prefect, or cloud-native solutions (e.g., AWS Step Functions, Azure Data Factory).
    b. Processing Engines: Use appropriate engines like Spark, Flink, or Dask for transformations.
    c. Storage Solutions: Optimize for speed and cost (e.g., S3 for object storage, Snowflake for analytics).
3. Data Quality
    a. Validation: Include checks to ensure data integrity (e.g., schema validation, null checks).
    b. Monitoring: Use tools like Great Expectations or Monte Carlo for monitoring data quality.
4. Security and Compliance
    a. Data Security: Encrypt data at rest and in transit.
    b. Access Control: Implement role-based access control (RBAC).
    c. Compliance: Adhere to GDPR, HIPAA, or other relevant regulations.
5. Scalability and Performance
    a. Resource Management: Monitor compute resources and optimize task parallelism.
    b. Autoscaling: Use cloud-native capabilities to scale infrastructure dynamically.
    c. Partitioning: Partition data to improve processing speed.
6. Error Handling and Recovery
    a. Idempotency: Design tasks to be idempotent, ensuring retries don't produce duplicate results.
    b. Checkpointing: Use checkpoints for long-running tasks to save progress.
7. Observability and Alerts
    a. Dashboards: Visualize pipeline performance and health.
    b. Alerting: Set up alerts for SLA breaches, failures, or performance issues.
    c. Logging: Use structured logs for troubleshooting.
8. Cost Optimization
    a. Resource Utilization: Optimize compute resources to reduce costs.
    b. Data Lifecycle: Implement tiered storage and data deletion policies.
9. Testing and CI/CD
    a. Pipeline Testing: Use unit, integration, and end-to-end tests to ensure reliability.
    b. Versioning: Version pipeline configurations and scripts.
    c. CI/CD: Automate deployment and updates to pipelines.
10. Governance and Auditability
    a. Lineage: Track data lineage to understand transformations and dependencies.
    b. Audit Logs: Keep logs of pipeline runs and changes for compliance and debugging.

Pipeline Failure

Pipeline Failure

Fail Gracefully:

- **Minimize Impact:** Isolate errors to prevent entire pipelines from failing.
- **Checkpointing:** Save progress at key stages to allow for restart without reprocessing.
- **Fallback Mechanisms:** Use default or placeholder data to maintain operational continuity.
- **Error Handling:** Log issues with detailed context for efficient debugging.

Fail Loudly:

- **Immediate Alerting:** Notify stakeholders or monitoring systems as soon as a failure occurs.
- **Clear Error Messages:** Provide specific details to quickly identify and address the root cause.
- **Monitoring Integration:** Use tools like logs and dashboards for real-time visibility into pipeline health.
- **Define Escalation Paths:** Ensure critical failures trigger appropriate responses from on-call teams.

Ensuring Data Pipeline Reliability

Ensuring Data Pipeline Reliability

- **Ensures Reliable Insights:** High-quality data is essential for accurate analysis and decision-making.
- **Prevents Pipeline Failures:** Early detection of data issues avoids downstream errors and costly reprocessing.
- **Maintains Data Integrity:** Quality checks validate that data meets defined standards and logical consistency.
- **Supports Compliance:** Ensures adherence to data governance and regulatory requirements.
- **Improves System Trust:** Automated alerting provides timely notifications, enhancing confidence in pipeline reliability.
- **Facilitates Quick Issue Resolution:** Alerts enable immediate action to address data anomalies or failures.

Ensuring Data Pipeline Reliability

Standalone Data-Quality DAG

- Good for checks on overall data integrity
- Checks on Database integrity
- Checks on the size and integrity of the MetadataDB

Integrated Checks During Table Creation

- Suggested for important data
- Can catch errors as they appear
- Gets Data Engineers in the habit of taking care of their own pipelines

Ensuring Data Pipeline Reliability

Types of Error Checks:

- **Integrity Checks**
  - Uniqueness
  - Null Values
  - Outliers
- **Domain Knowledge Checks**
  - Value Bounds
  - Value Categories

Ensuring Data Pipeline Reliability

Pipeline Failure:

- Should the pipeline fail when an error is hit
- Should the pipeline continue and just alert

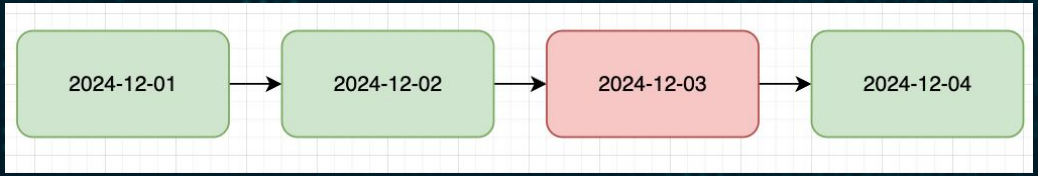
Alerting:

- What level of alerting should be used:
  - **Catastrophic -** Alerts loudly at all times of day
  - **High -** Alerts loudly during work hours
  - **Warning -** Alerts quietly
- Should you connect an Incident Management platform like PagerDuty?

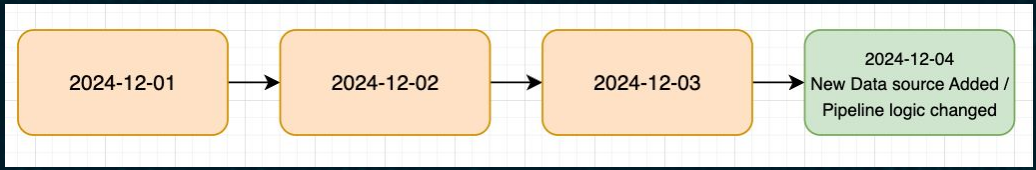
Backfilling Pipelines

Backfilling Pipelines

Data Failed on a specific day



New Data or Code Introduced and we want to retroactively change data



Backfilling Pipelines

Define the Scope:

- Identify the time range or specific data partitions to backfill.
- Assess which pipeline components or data sources require backfilling.

Evaluate Data Volume:

- Estimate the volume of data to process and ensure the infrastructure can handle it.
- Split large backfills into manageable chunks to avoid resource bottlenecks.

Data Dependencies:

- Verify that upstream datasets or services needed for backfilling are complete and consistent.
- Ensure downstream systems can handle the influx of backfilled data without disruption.

Performance Optimization:

- Prioritize critical data first using parallelization or partitioned runs.
- Avoid overloading shared resources by scheduling backfills during off-peak hours.

Backfilling Pipelines

Handle Duplicates:

- Ensure idempotency by de-duplicating data or overwriting existing records cleanly.
- Verify that the backfill process doesn't introduce redundant entries.

Monitor and Validate:

- Use logging, alerts, and checkpoints to track progress and identify issues.
- Perform data quality checks to validate the integrity of backfilled data.

Communicate with Stakeholders:

- Inform teams about potential delays or impacts on downstream systems.
- Coordinate with data consumers to confirm expected outcomes.

Test First:

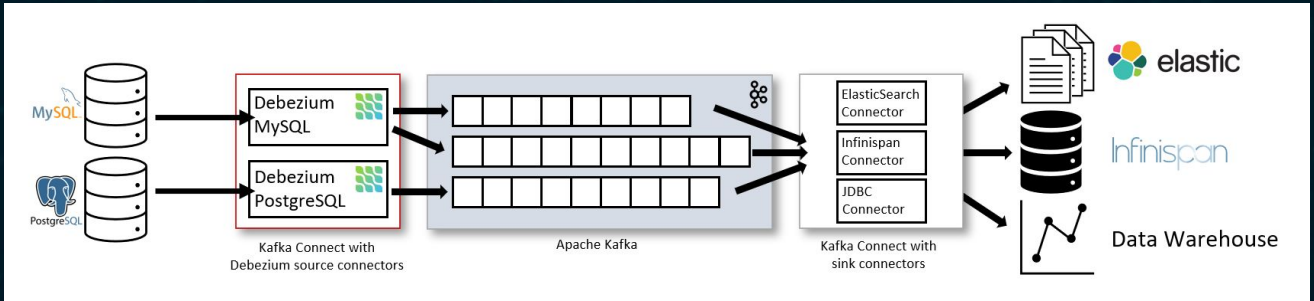
- Run small-scale test backfills in a staging environment to uncover potential issues.
- Validate transformations, data quality checks, and error handling before full execution.

Change Data Capture

Change Data Capture

A Change Data Capture (CDC) data pipeline enables the continuous tracking of changes (inserts, updates, deletes) to data in a source system (e.g., a database) and propagates those changes downstream to target systems (e.g., data lakes, warehouses, or analytics platforms). Using Debezium simplifies the CDC implementation by leveraging database logs to capture changes non-intrusively.

1. Source Database:
   a. The database where changes occur (e.g., MySQL, PostgreSQL, MongoDB).
   b. Must support transaction logs (e.g., WAL for PostgreSQL, binlog for MySQL).
2. Debezium:
   a. An open-source CDC platform that reads database logs to detect changes.
   b. Outputs changes as events in a standardized format (e.g., Avro, JSON).
3. Message Broker:
   a. Typically, Kafka is used to transport the change events produced by Debezium.
   b. Other brokers like Pulsar or RabbitMQ can be integrated with additional setup.
4. Consumers:
   a. Systems or services that process and apply the captured changes, such as:
   b. Data warehouses (e.g., Snowflake, BigQuery, Redshift).
   c. Data lakes (e.g., AWS S3, Azure Data Lake).
   d. Real-time analytics platforms or microservices.



Security in the Cloud

Building Pipelines in the Cloud

Building Pipelines in the Cloud

- Orchestration Services
  - AWS Managed Workflows for Apache Airflow (MWAA):
    - Fully managed Airflow service for building and orchestrating pipelines.
    - Supports complex workflows with tasks defined as Directed Acyclic Graphs (DAGs).
    - Integrates seamlessly with AWS services (S3, Redshift, Glue, etc.).
    - Best for when you need Python-based control and flexibility.
  - Azure Data Factory (ADF):
    - A fully managed, low-code orchestration service.
    - Provides an intuitive drag-and-drop interface for designing pipelines.
    - Supports data ingestion, transformation, and orchestration across on-premises and cloud systems.
    - Best for large-scale ETL and hybrid cloud integration.
  - Google Cloud Composer
    - Managed Apache Airflow on Google Cloud.
    - Similar to MWAA but optimized for GCP services.

Building Pipelines in the Cloud

- ETL/ELT Tools
  - AWS Glue:
    - Serverless ETL service for data transformation and loading.
    - Includes Glue Data Catalog for metadata management.
    - Supports Python-based transformations using PySpark.
    - Best for preprocessing large datasets stored in S3 for analysis in Redshift or Athena.
  - Azure Data Factory:
    - Includes built-in data transformation capabilities via data flows.
    - Supports scalable data movement using native connectors for Azure Blob Storage, Azure Synapse, and more.
  - Google Cloud Dataflow:
    - Serverless data processing service using Apache Beam.
    - Great for batch and streaming data transformations.

Building Pipelines in the Cloud

- Data Storage and Warehousing
  - Amazon S3:
    - Scalable object storage for staging and archiving data.
    - Often used as a landing zone for raw data.
  - Amazon Redshift:
    - Data warehouse solution for analytics workloads.
    - Works seamlessly with AWS Glue for data loading.
  - Azure Blob Storage:
    - Scalable storage solution for unstructured data.
    - Acts as a staging area for ADF pipelines.
  - Azure Synapse Analytics:
    - Integrated data analytics and warehousing platform.
    - Combines SQL and big data capabilities for analytics at scale.
  - Google BigQuery:
    - Fully managed, serverless data warehouse.
    - Excels at fast querying over large datasets.

Building Pipelines in the Cloud

- Monitoring and Logging
  - AWS CloudWatch
    - Centralized monitoring for Glue jobs, MWAA, and Redshift.
  - Azure Monitor
    - Tracks the performance and health of Data Factory pipelines.
  - Google Cloud Monitoring
    - Provides insights into the performance of Dataflow and Composer pipelines.

Cost Management in the Cloud

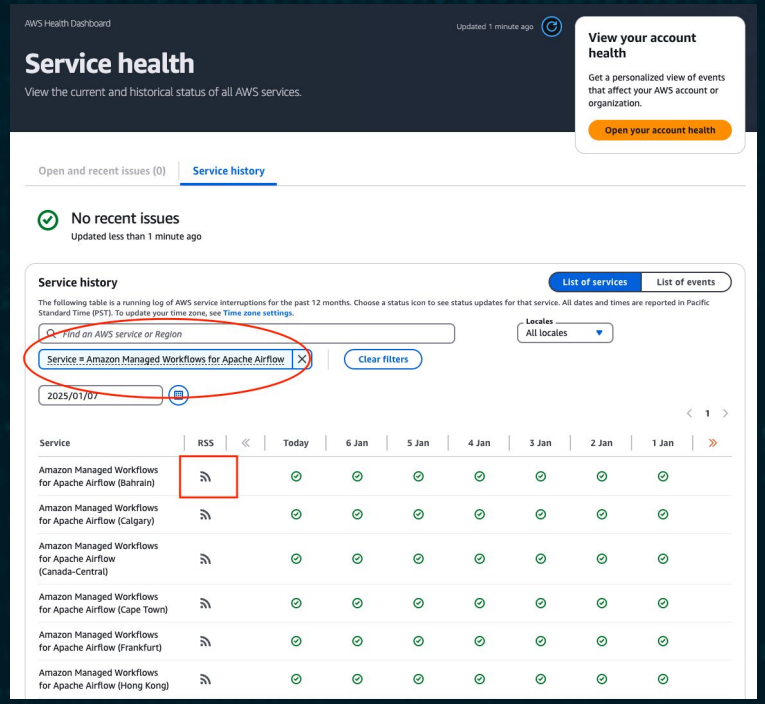
Cost Management in the Cloud

- Optimize Log Generation
  - Log Level Management:
    - Use appropriate log levels (ERROR, WARN, INFO, DEBUG) based on the environment.
    - Avoid verbose logging (e.g., DEBUG or TRACE) in production unless troubleshooting specific issues.
  - Filter Logs at the Source:
    - Collect only relevant logs instead of every detail.
    - Use structured logging (e.g., JSON format) to make logs more efficient for parsing and filtering.
- Control Log Retention
  - Retention Policies:
    - Set retention policies to automatically delete older logs that are no longer needed.
    - Keep short retention periods for high-volume logs and archive critical logs to cheaper storage.
  - Archive Logs to Cheaper Storage:
    - Move older logs to cost-effective storage solutions like:
      - AWS S3 Glacier
      - Azure Archive Blob
      - Google Cloud Archive
- Use Log Analytics Judiciously
  - Pay-as-You-Go vs. Reserved Capacity:
  - For log analytics services (e.g., AWS CloudWatch Logs Insights, Azure Monitor Logs, Google Cloud Logging), assess whether pay-as-you-go or reserved capacity is more cost-effective.
- Query Optimization:
  - Avoid running frequent, expensive queries on large datasets.
  - Use filters and aggregated metrics to reduce query costs.
- Monitor Log Costs
  - Alert on Log Usage:
    - Set up alerts when log storage or analytics usage exceeds predefined thresholds.
  - Use Cost Analysis Tools:
    - Leverage tools like AWS Cost Explorer, Azure Cost Management, or Google Cloud Billing to track costs from logging services.

Cost Management in the Cloud

- Right-Sizing Resources
  - Analyze Workload Requirements:
    - Use monitoring tools to analyze CPU, memory, and storage utilization.
    - Downgrade resources if utilization is consistently low.
  - Autoscaling:
    - Enable autoscaling for compute services (e.g., EC2, Azure VMs, GKE, or AKS) to automatically adjust resources based on demand.
- Use Serverless or Managed Services
  - Serverless Compute:
    - Services like AWS Lambda, Azure Functions, and Google Cloud Functions scale automatically and charge only for usage, eliminating overprovisioning.
  - Managed Databases:
    - Services like AWS Aurora Serverless or Azure SQL Hyperscale dynamically scale storage and compute based on load.
    - Leverage Spot or Preemptible Instances
  - Non-Critical Workloads:
    - Use spot instances (AWS) or preemptible VMs (GCP) for tasks like batch processing, which tolerate interruptions and cost significantly less.
- Resource Allocation Policies
  - Instance Families:
    - Choose appropriate instance types for workloads (e.g., memory-optimized for databases, compute-optimized for AI/ML tasks).
  - Quotas and Limits:
    - Set quotas to prevent provisioning of excessive resources.
- Regular Audits
  - Idle Resources:
    - Identify and shut down idle resources, such as unused VMs, over-provisioned storage volumes, and underutilized load balancers.
  - Scheduled Scaling:
    - Use schedules to scale down resources during non-peak hours (e.g., development/test environments).

Managing Outages in the Cloud

Managing Outages in the Cloud

- Advance Alerting
  - Sign up for RSS alerts from cloud providers
  - Have alerting for Engineers ready
  - Do a system check in the morning
- Communication to Stakeholders
  - Clear communication will build trust in your org
  - Have some way of knowing who the users of a pipeline or table are for easy communication